

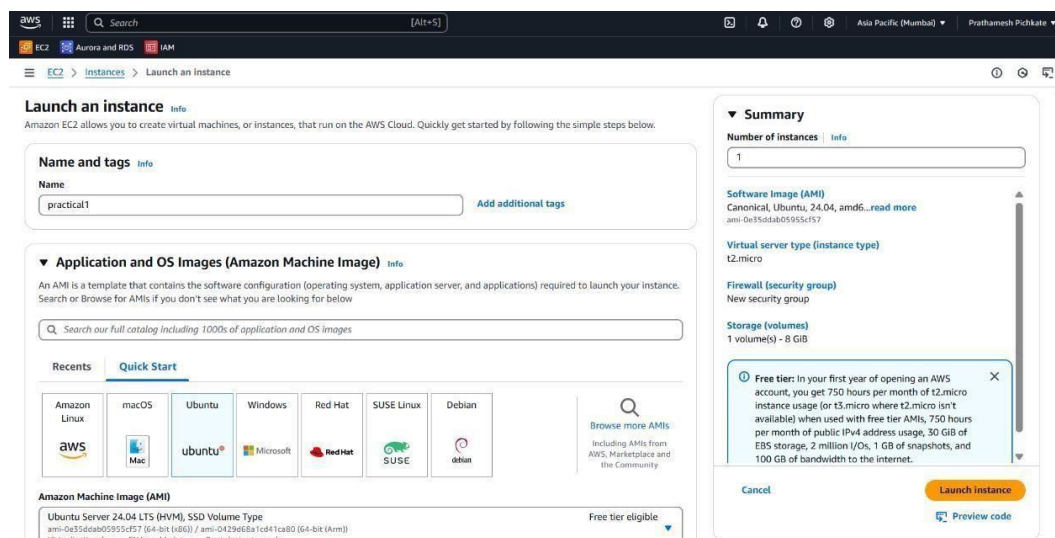
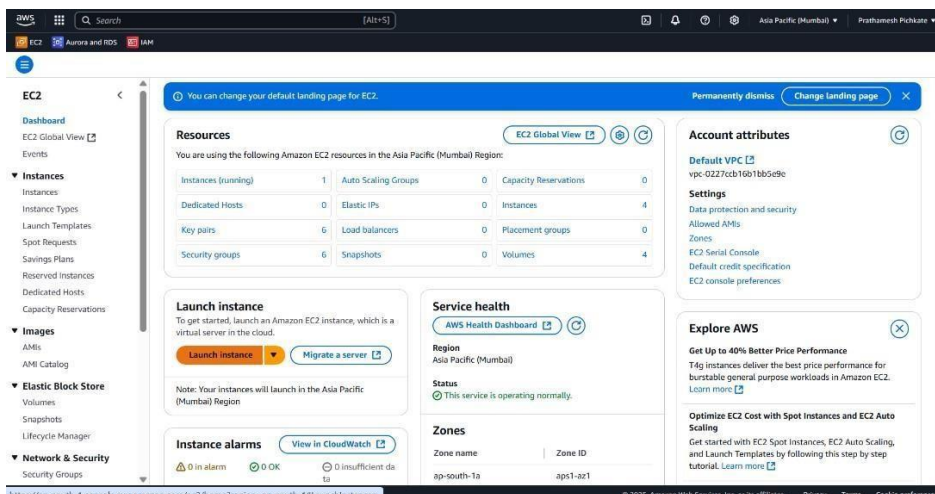
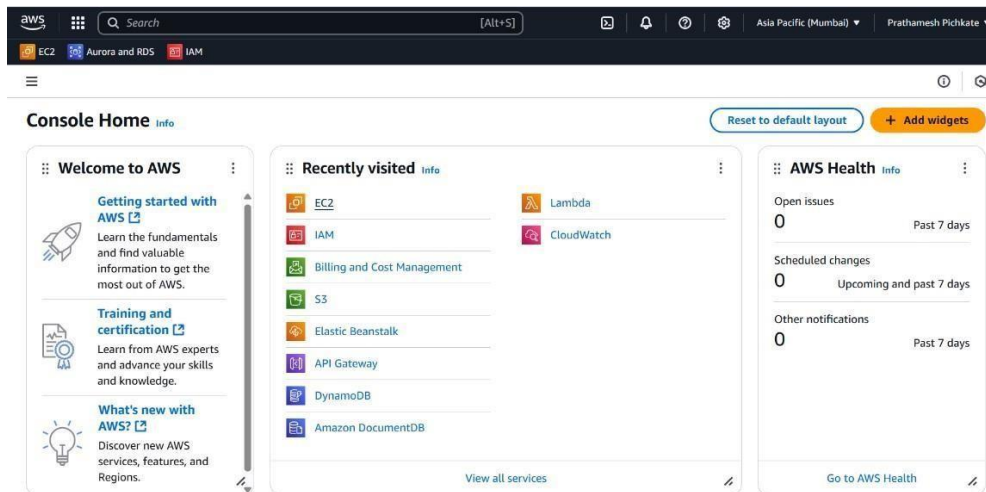
CC - Practical 1

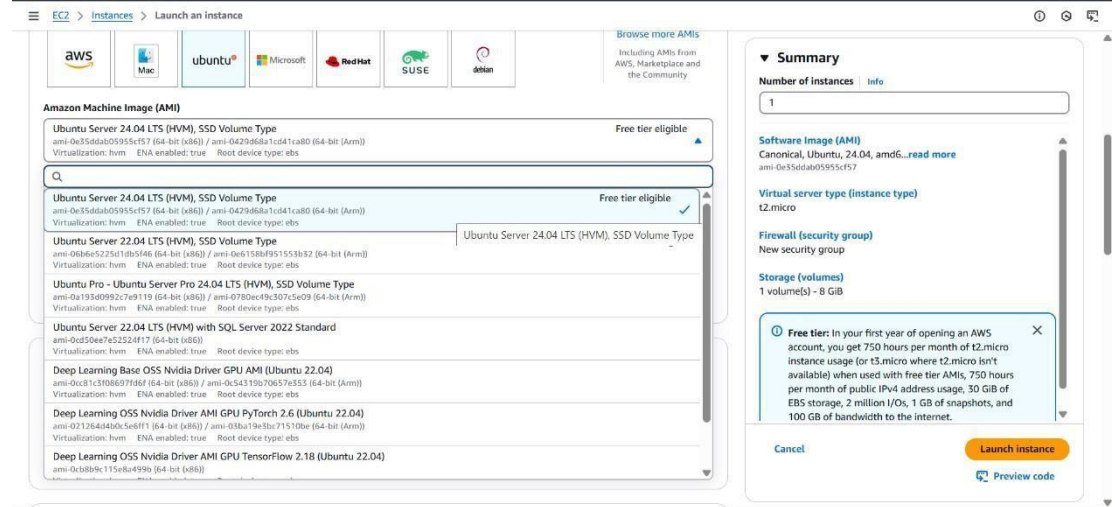
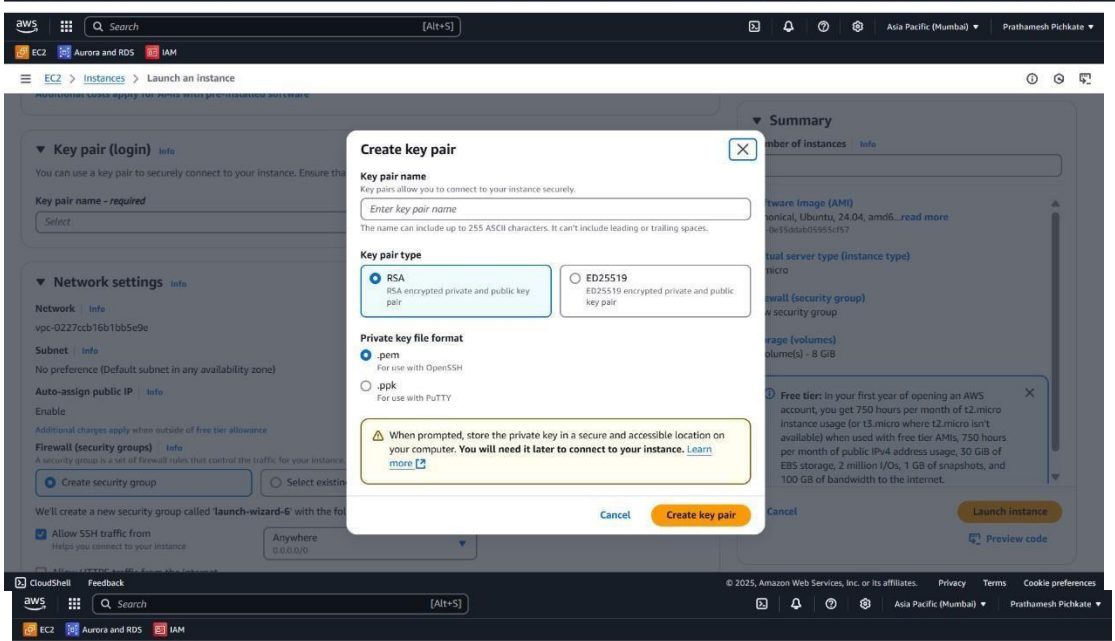
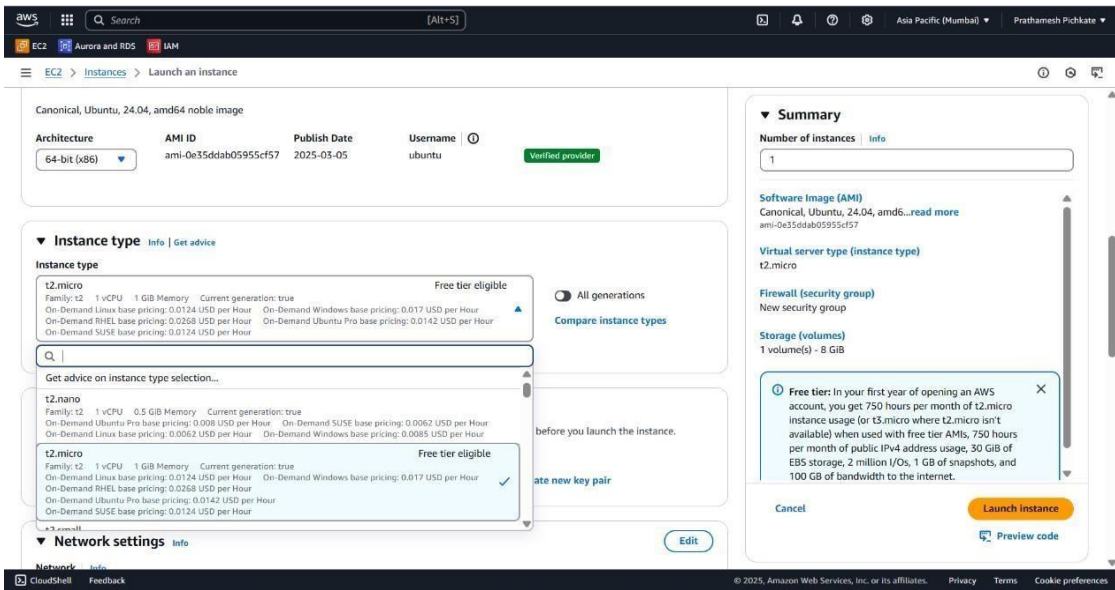
Name:- Ganesh Malaji Ghule

Roll No.:- 37

Div:- TE-A

Title:- Case Study on amazon EC2





EC2

Aurora and RDS

IAM

Search

[Alt+S]

Asia Pacific (Mumbai)

Prathamesh Pichkate

EC2

Instances

Launch an instance

Allow HTTP traffic from the internet

To set up an endpoint, for example when creating a web server

Rules with source of 0.0.0.0/0 allow all IP addresses to access your instance. We recommend setting security group rules to allow access from known IP addresses only.

Configure storage

Info

Advanced

1x 8 GiB gp3 Root volume, 3000 IOPS, Not encrypted

Free tier eligible customers can get up to 30 GB of EBS General Purpose (SSD) or Magnetic storage

Add new volume

Click refresh to view backup information

The tags that you assign determine whether the instance will be backed up by any Data Lifecycle Manager policies.

0 x File systems

Edit

Advanced details

Info

Summary

Number of instances

Info

1

Software Image (AMI)

Canonical, Ubuntu, 24.04, amd64...

read more

ami-0e35dab05955c157

Virtual server type (instance type)

t2.micro

Firewall (security group)

New security group

Storage (volumes)

1 volume(s) - 8 GiB

Free tier: In your first year of opening an AWS account, you get 750 hours per month of t2.micro instance usage (or t3.micro where t2.micro isn't available) when used with free tier AMIs, 750 hours per month of public IPv4 address usage, 30 GiB of EBS storage, 2 million I/Os, 1 GiB of snapshots, and 100 GB of bandwidth to the internet.

Cancel

Launch instance

Preview code

EC2

Aurora and RDS

IAM

Search

[Alt+S]

Asia Pacific (Mumbai)

Prathamesh Pichkate

EC2

Instances

Launch an instance

Success

Successfully initiated launch of instance (i-01af0b197619da280)

Launch log

Next Steps

What would you like to do next with this instance, for example "create alarm" or "create backup"

1

2

3

4

5

6

Create billing and free tier usage alerts

To manage costs and avoid surprise bills, set up email notifications for billing and free tier usage thresholds.

Create billing alerts

Connect to your instance

Once your instance is running, log into it from your local computer.

Connect to instance

Learn more

Connect an RDS database

Configure the connection between an EC2 instance and a database to allow traffic flow between them.

Connect an RDS database

Create a new RDS database

Learn more

Create EBS snapshot policy

Create a policy that automates the creation, retention, and deletion of EBS snapshots

Create EBS snapshot policy

Manage detailed monitoring

Enable or disable detailed monitoring for the instance. If you enable detailed monitoring, the Amazon EC2 console displays monitoring graphs with a 1-minute period.

Create Load Balancer

Create an application, network gateway or classic Elastic Load Balancer

Create Load Balancer

Create AWS budget

AWS Budgets allows you to create budgets, forecast spend, and take action on your costs and usage from a single location.

Manage CloudWatch alarms

Create or update Amazon CloudWatch alarms for the instance.

Manage CloudWatch alarms

EC2

Aurora and RDS

IAM

Search

[Alt+S]

Asia Pacific (Mumbai)

Prathamesh Pichkate

EC2

Instances

Launch an instance

Network settings

Info

Edit

Network

Info

vpc-0227cb16b1bb5e9e

Subnet

Info

No preference (Default subnet in any availability zone)

Auto-assign public IP

Info

Enable

Additional charges apply when outside of free tier allowance

Firewall (security group)

Info

Create security group

Select existing security group

We'll create a new security group called "launch-wizard-6" with the following rules:

Allow SSH traffic from

Helps you connect to your instance

Anywhere

0.0.0.0/0

Allow HTTPS traffic from the internet

To set up an endpoint, for example when creating a web server

Allow HTTP traffic from the internet

To set up an endpoint, for example when creating a web server

Rules with source of 0.0.0.0/0 allow all IP addresses to access your instance. We recommend setting security group rules to allow access from known IP addresses only.

Configure storage

Info

Advanced

Summary

Number of instances

Info

1

Software Image (AMI)

Canonical, Ubuntu, 24.04, amd64...

read more

ami-0e35dab05955c157

Virtual server type (instance type)

t2.micro

Firewall (security group)

New security group

Storage (volumes)

1 volume(s) - 8 GiB

Free tier: In your first year of opening an AWS account, you get 750 hours per month of t2.micro instance usage (or t3.micro where t2.micro isn't available) when used with free tier AMIs, 750 hours per month of public IPv4 address usage, 30 GiB of EBS storage, 2 million I/Os, 1 GiB of snapshots, and 100 GB of bandwidth to the internet.

Cancel

Launch instance

Preview code

CC Practical -2

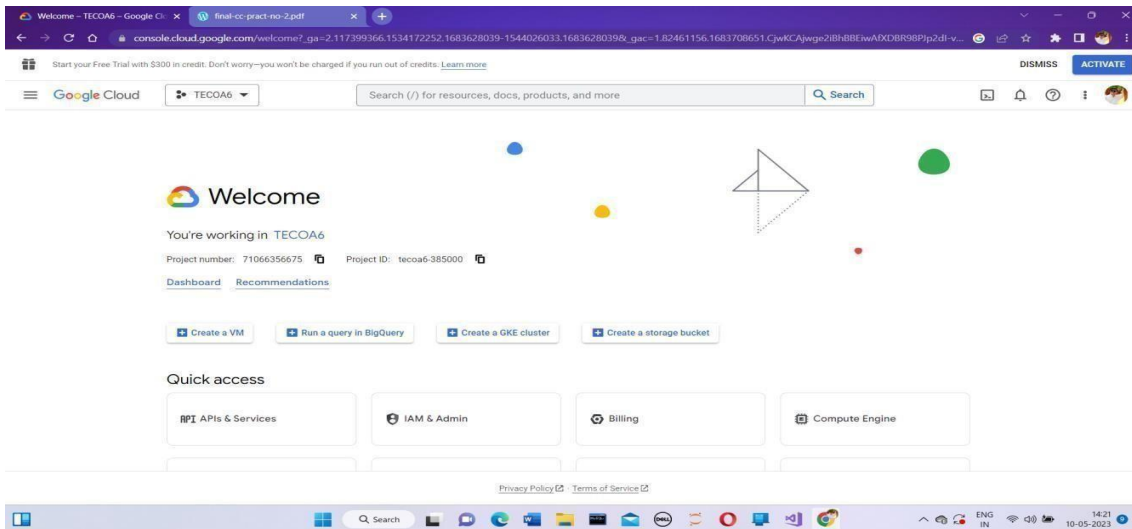
Name:- Ganesh Malaji Ghule

Roll No.:- 37

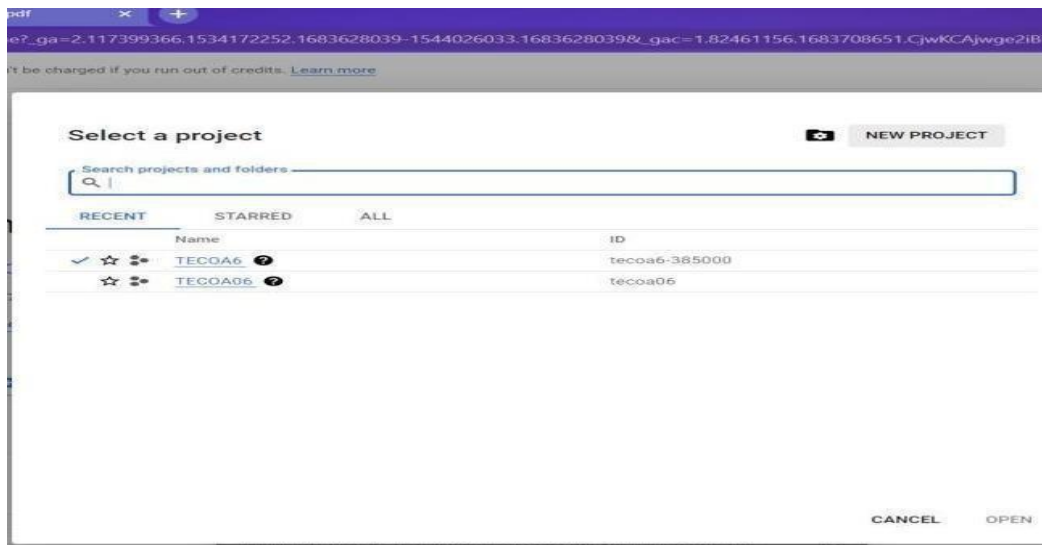
Div:- TE-A

Title:- Installation and configure Google app engine

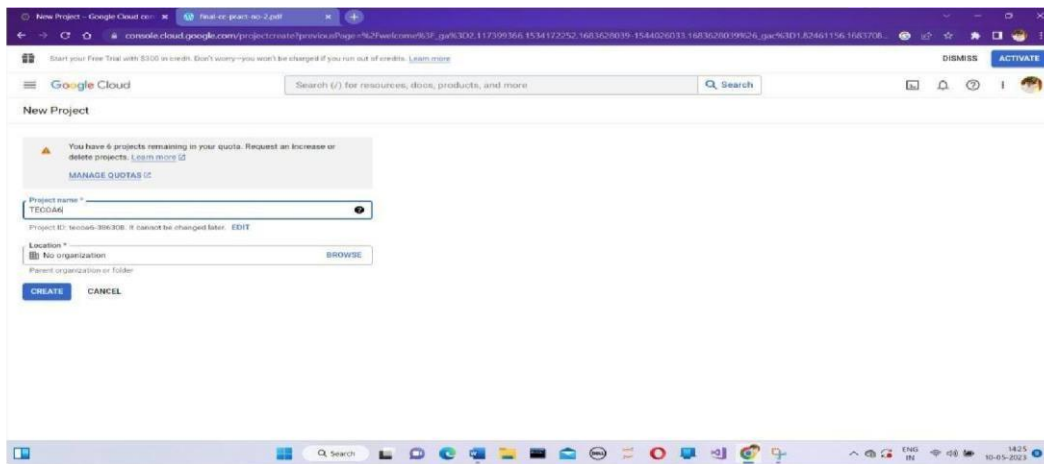
Step 1:- Search Google Cloud Platform in a any search engine& Click on Console.



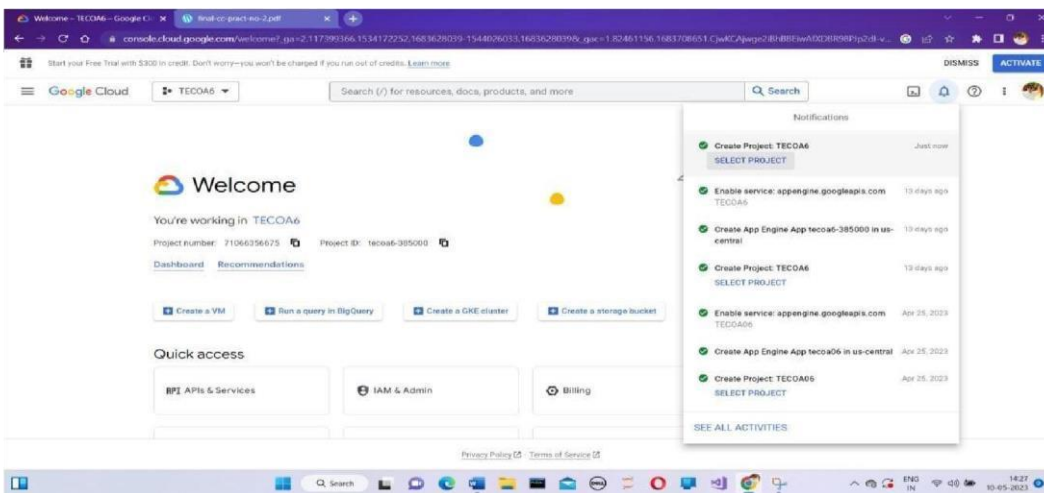
Step 2:- Click on select new project.



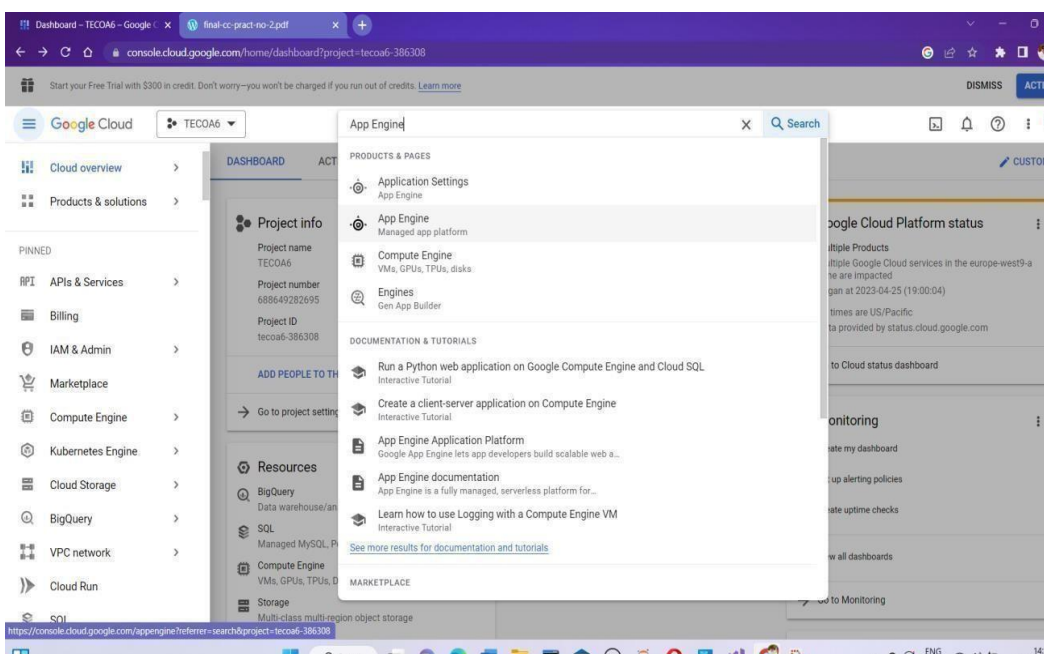
Step 3: Give project name and click on create



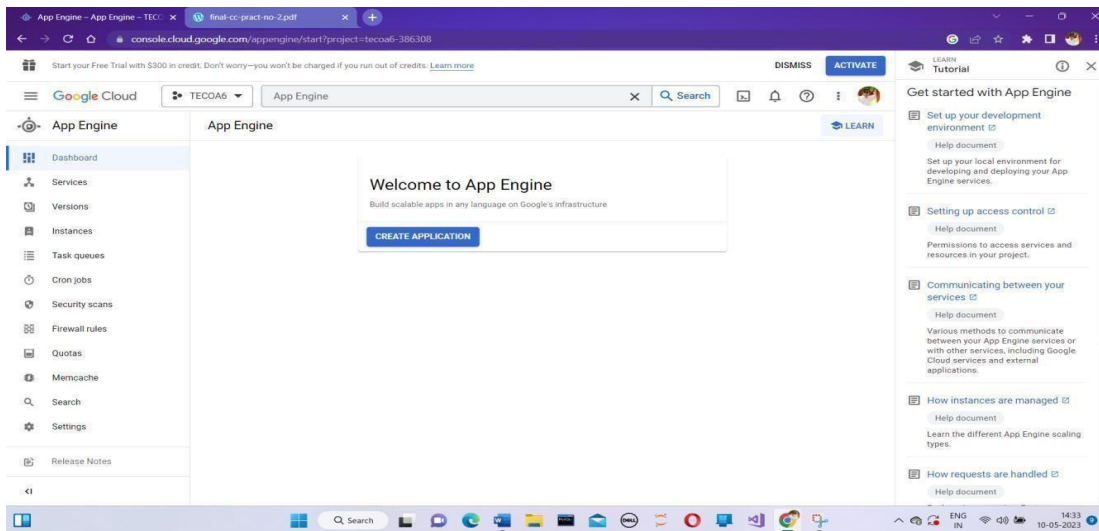
Step4:- Click on select project



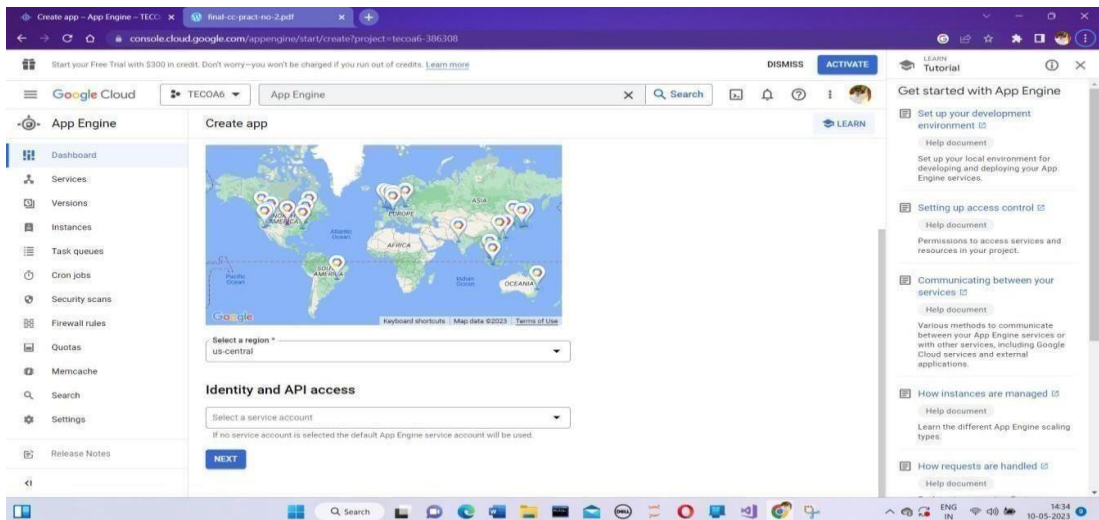
Step5:- In a search bar type search App Engine



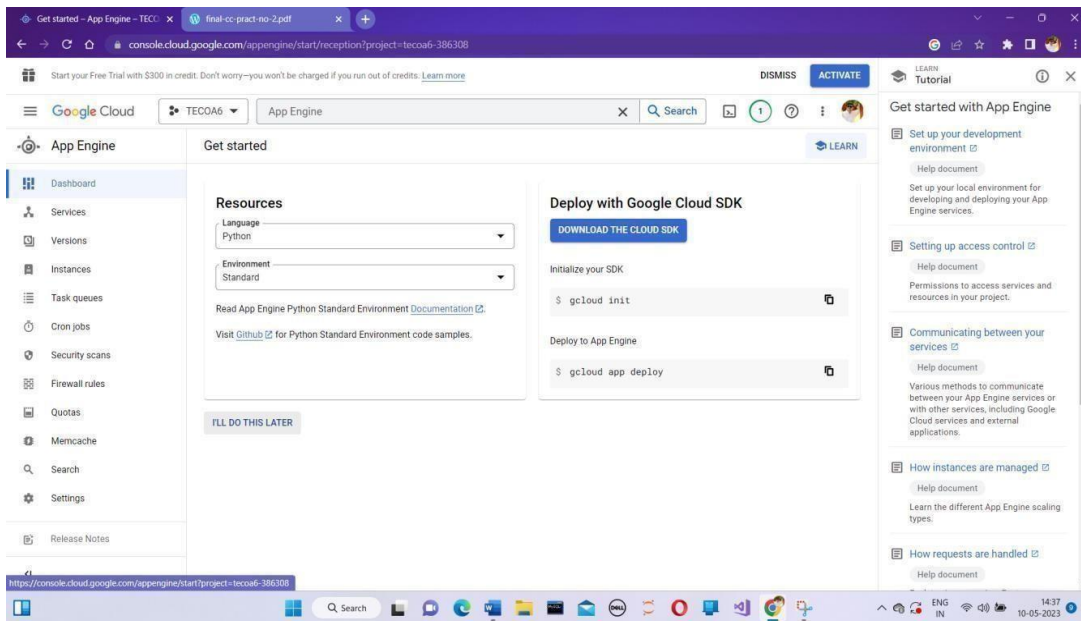
Step 6:- Click on App Engine and Following screen will appear& Click on Create Application.



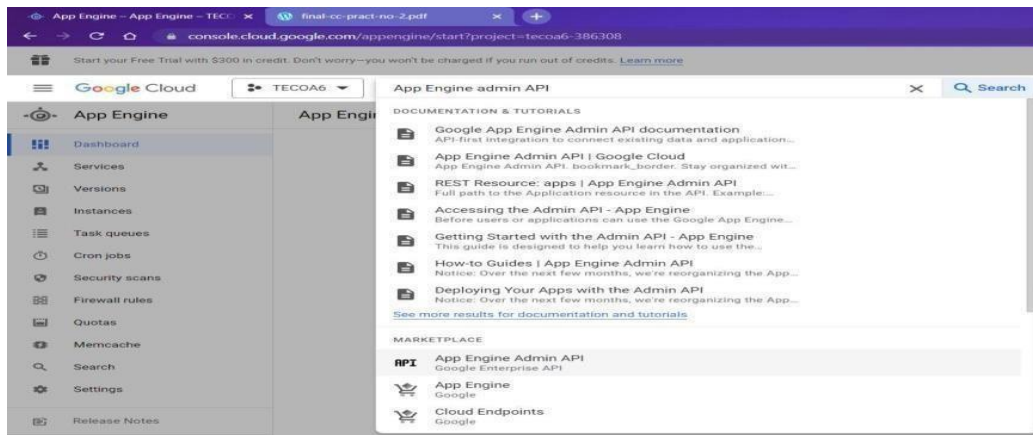
Step 7 : Click on next.



Step 8:- Scroll down and click on I'll do this later.

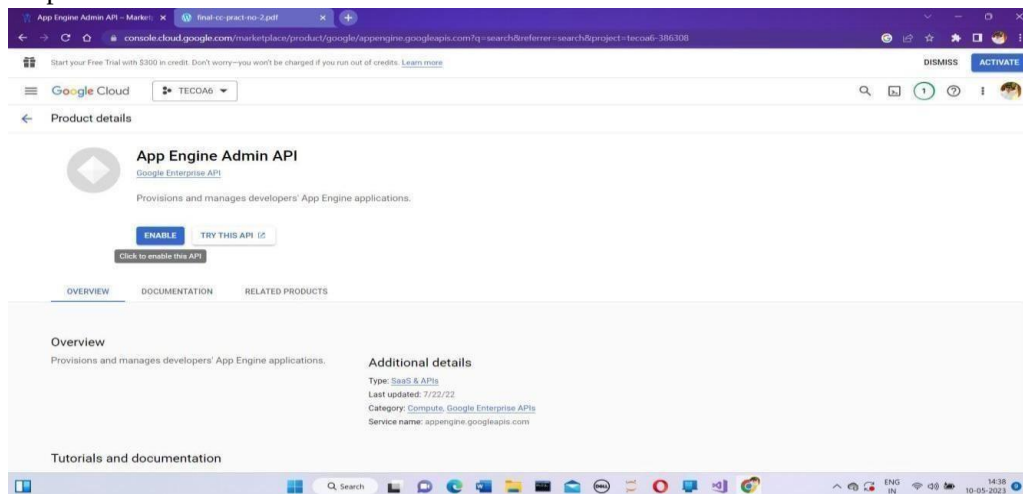


Step 9:- In search bar type App Engine Admin API.

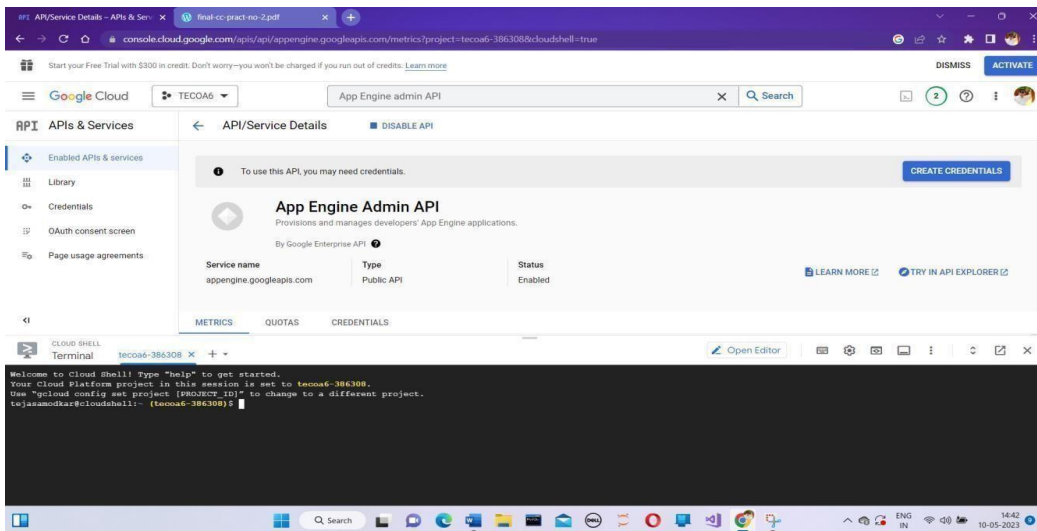


Step 11:- Click Activate Cloud Shell:-

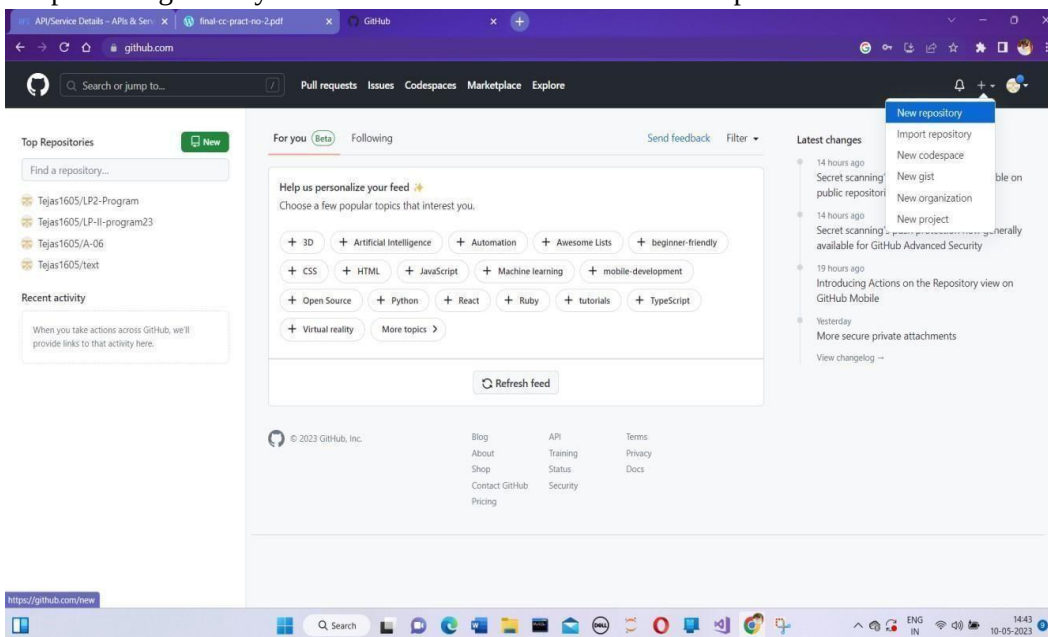
Step 10:- Click Enables



Step 11:- Click Activate Cloud Shell. Following screen will appear:-

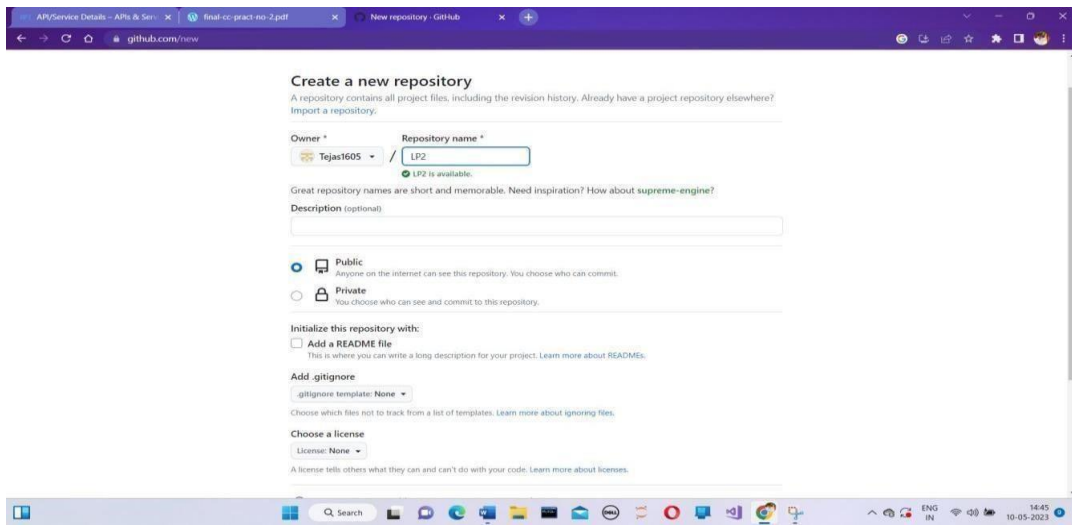


Step 13:- Login into your GitHub account and click on new repositor

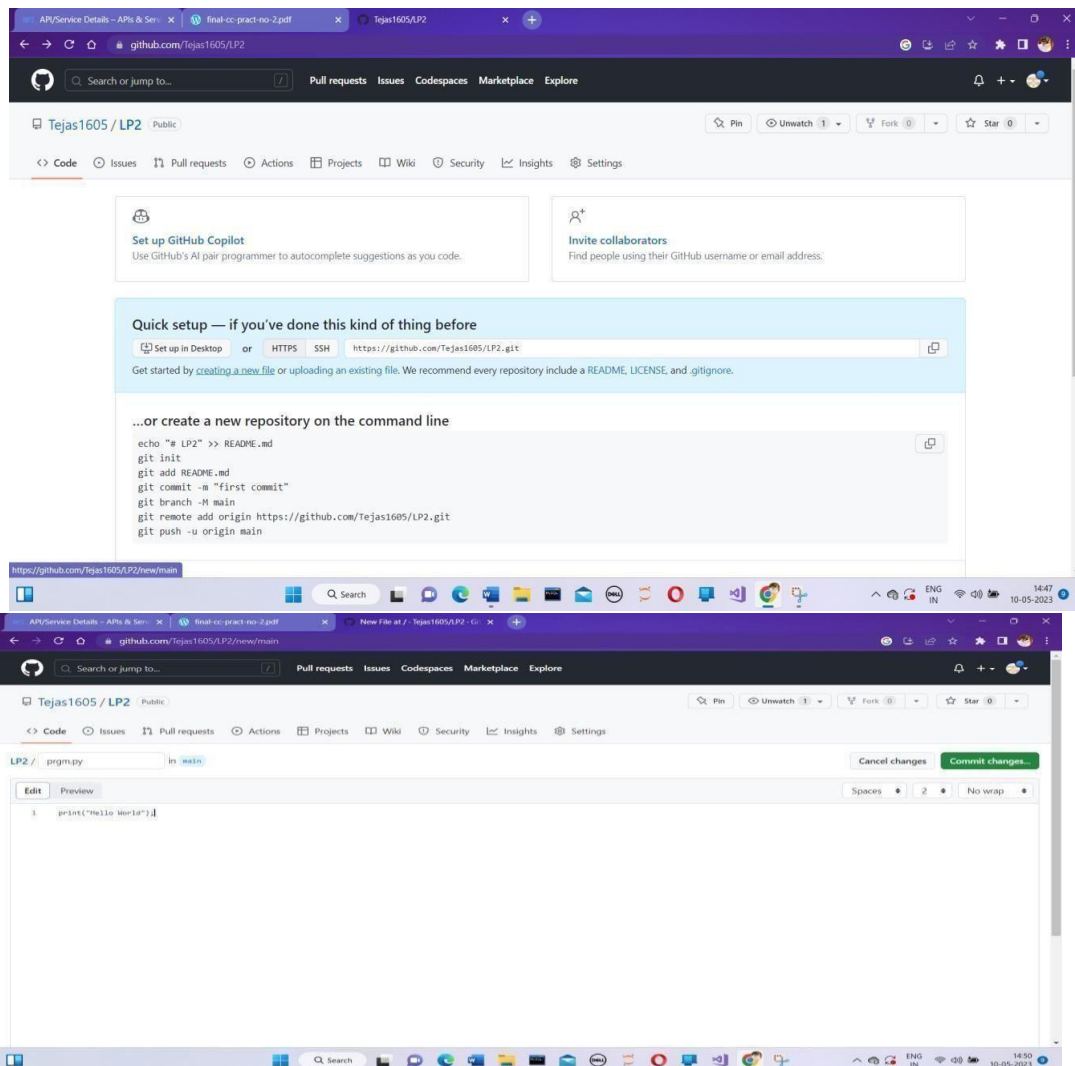


Step

14 :- Give name to your repository and click create.

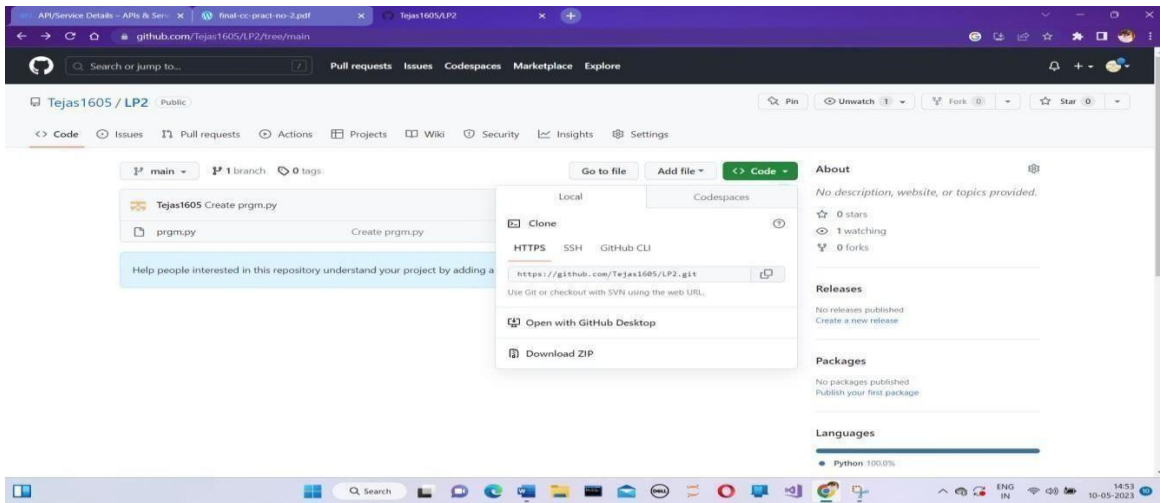


Step 15:- Click on creating new file.

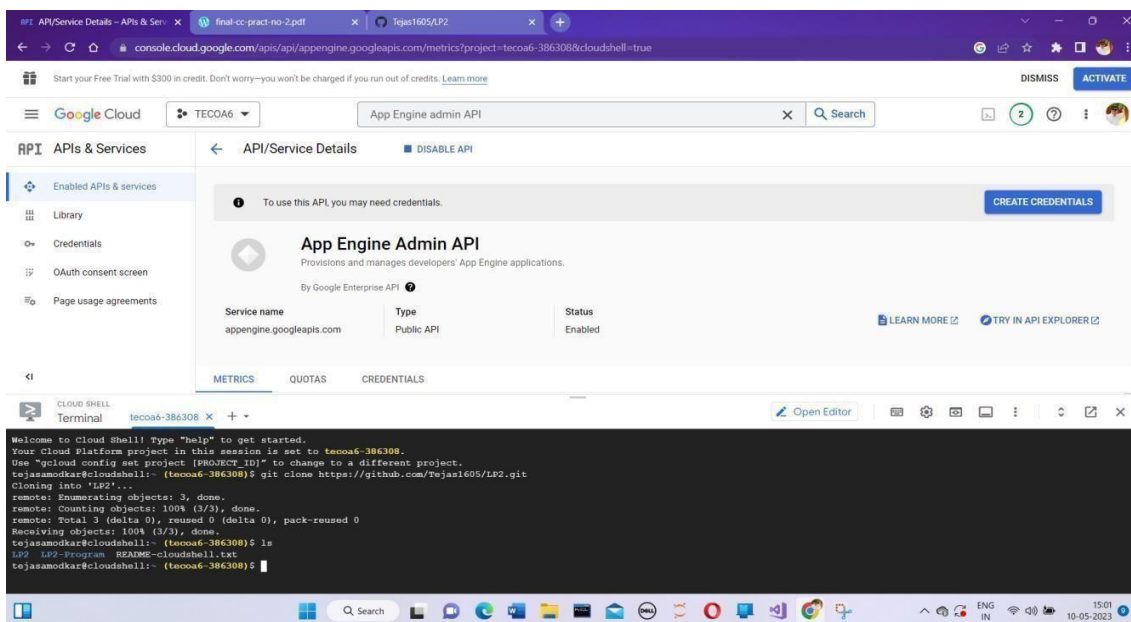


Step

Step 16 :- Give name to the python file & type your code. 17:- Click on code and copy URL

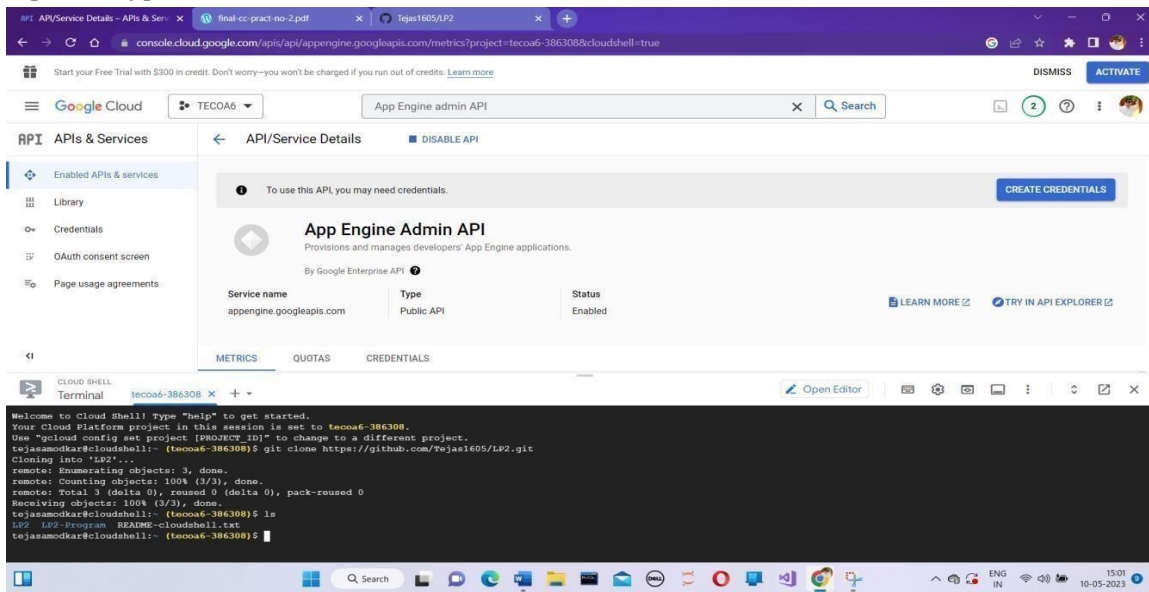


Step 18:- Go to cloud and type – git clone and paste url.

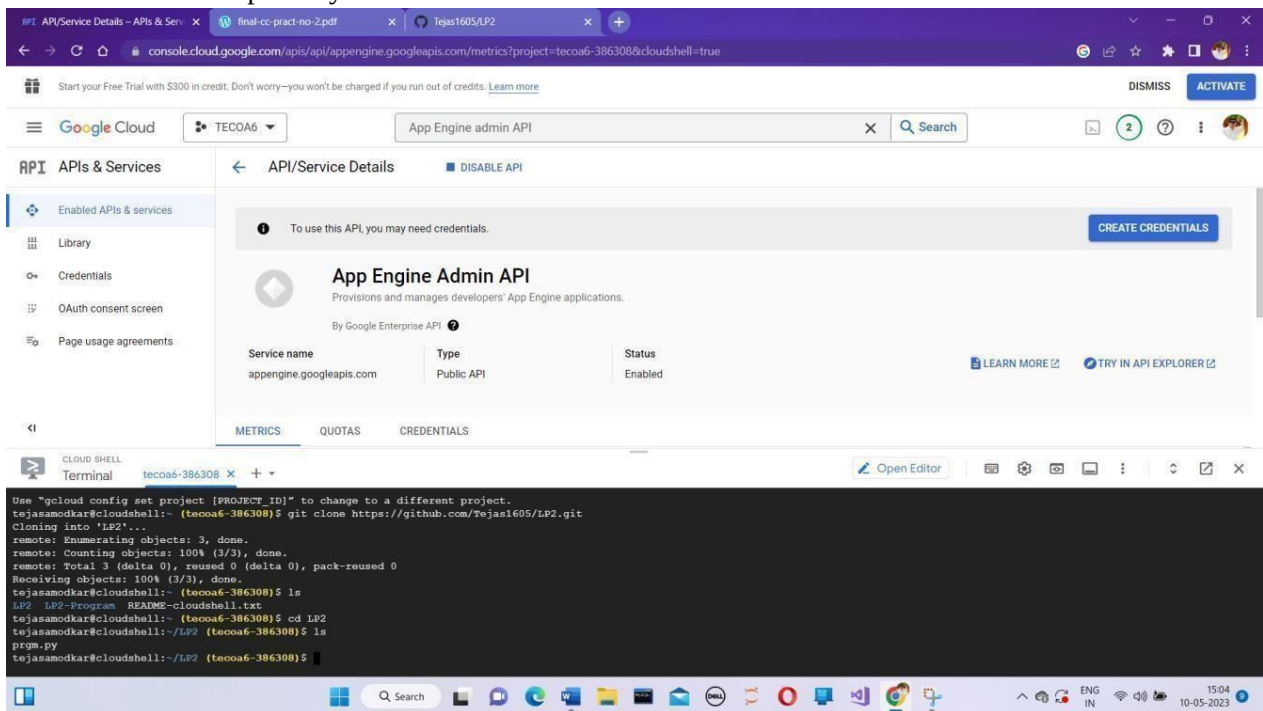


Step

Step 19 :- Type ls.



20 :- Enter cd-repository name.



Step

Step 35:- Type ls and to run python code type python-program name.

The screenshot displays the Google Cloud Console interface. The top navigation bar shows the Google Cloud logo, a search bar, and a user profile icon. The left sidebar contains navigation links for APIs & Services, Library, Credentials, OAuth consent screen, and Page usage agreements. The main content area is titled 'API/Service Details' and shows the 'App Engine Admin API' with a 'CREATE CREDENTIALS' button. Below this, a table lists the service name 'appengine.googleapis.com', type 'Public API', and status 'Enabled'. The bottom section of the console shows a terminal window with the following commands and output:

```
Cloning into 'LP2'...
remote: Enumerating objects: 3, done.
remote: Counting objects: 100% (3/3), done.
remote: Total 3 (delta 0), reused 0 (delta 0), pack-reused 0
Receiving objects: 100% (3/3), done.
tejasmodkar@cloudshell:~ (tecoa6-386308) $ ls
LP2 LP2-Program README-cloudshell.txt
tejasmodkar@cloudshell:~ (tecoa6-386308) $ cd LP2
tejasmodkar@cloudshell:~/LP2 (tecoa6-386308) $ ls
prgm.py
tejasmodkar@cloudshell:~/LP2 (tecoa6-386308) $ python prgm.py
Hello World
tejasmodkar@cloudshell:~/LP2 (tecoa6-386308) $
```


CC Practical -3

Name : Ganesh Malaji Ghule

Roll no : 37

Div : TE-A

Title : Creating an Application in Salesforce.com using Apex programming Language.

STEPS TO CREATE AN APPLICATION :

Step No 1: Create new org: <https://developer.salesforce.com/signup>

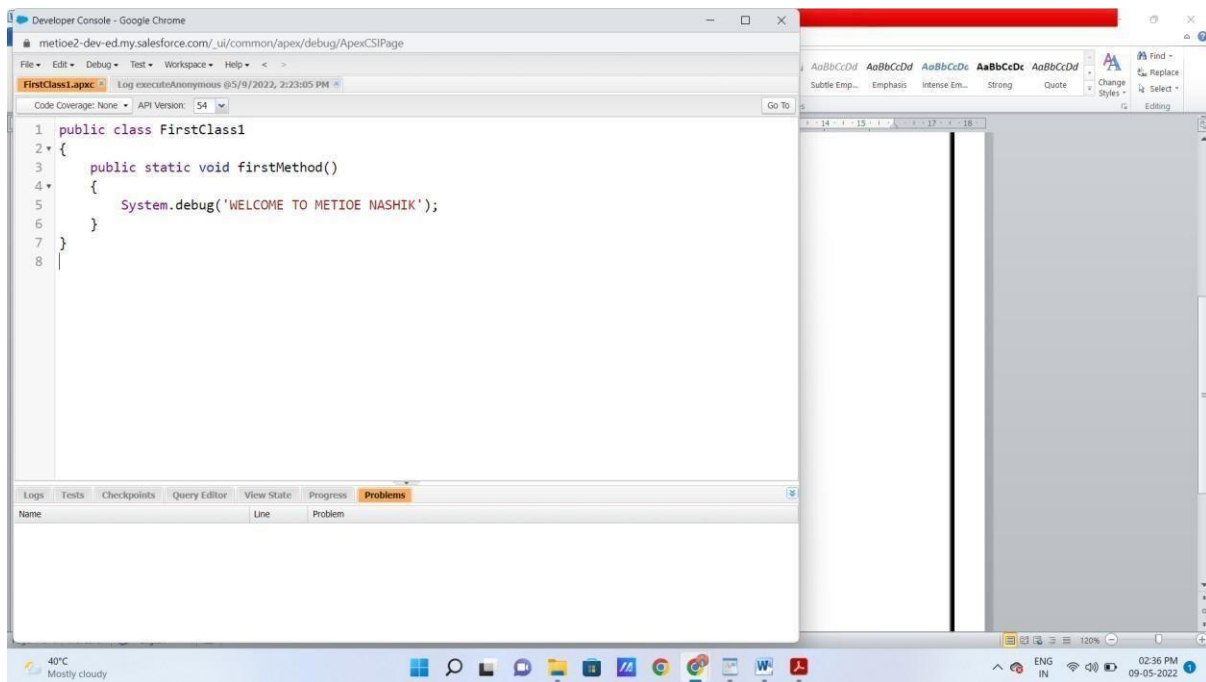
Step No 2: After signup, login using following URL <https://login.salesforce.com/>

Step No 3: Login Page (Enter your credential to login)

1. Open Developer Console
2. File ---> New ---> Select Apex Class
3. Type below mentioned code

```
public class FirstClass1
{
    public static void firstMethod()
    {
        System.debug('WELCOME TO METIOE NASHIK');
    }
}
```

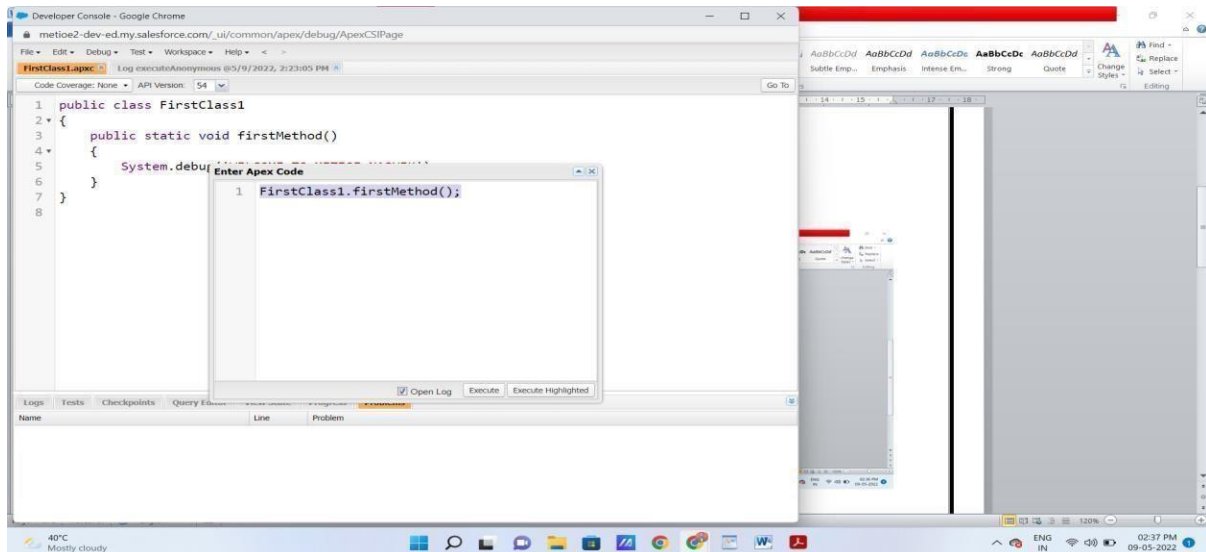
Note : you can write any code as application



4. Click on Debug ----> Open Execute Anonymous Window

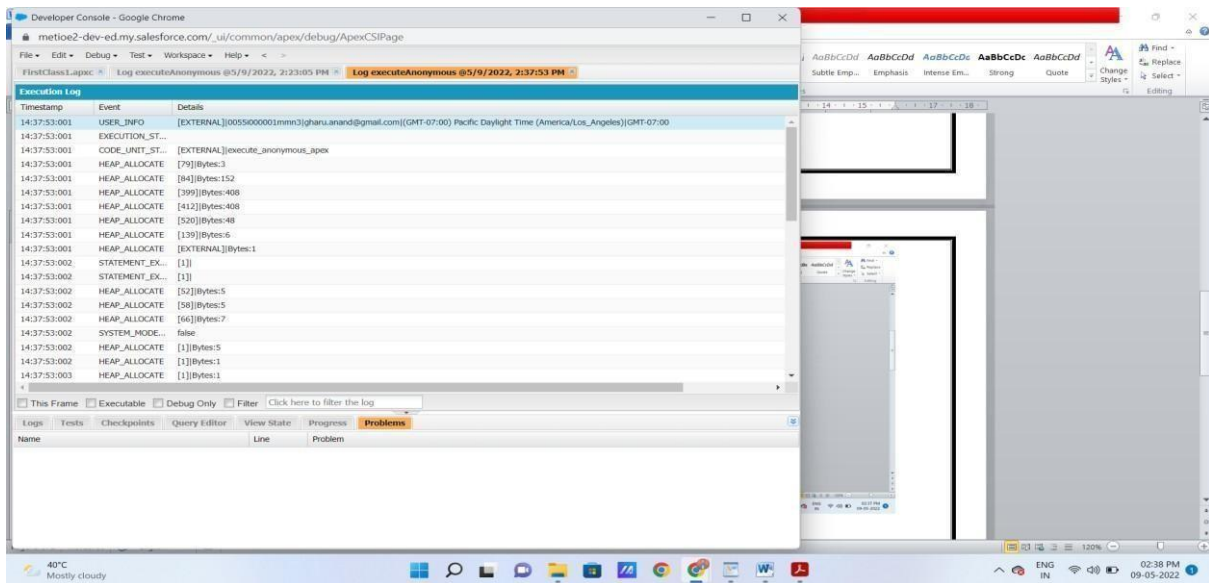
5. Type below code(Apex Code)

`FirstClass1.firstMethod();`

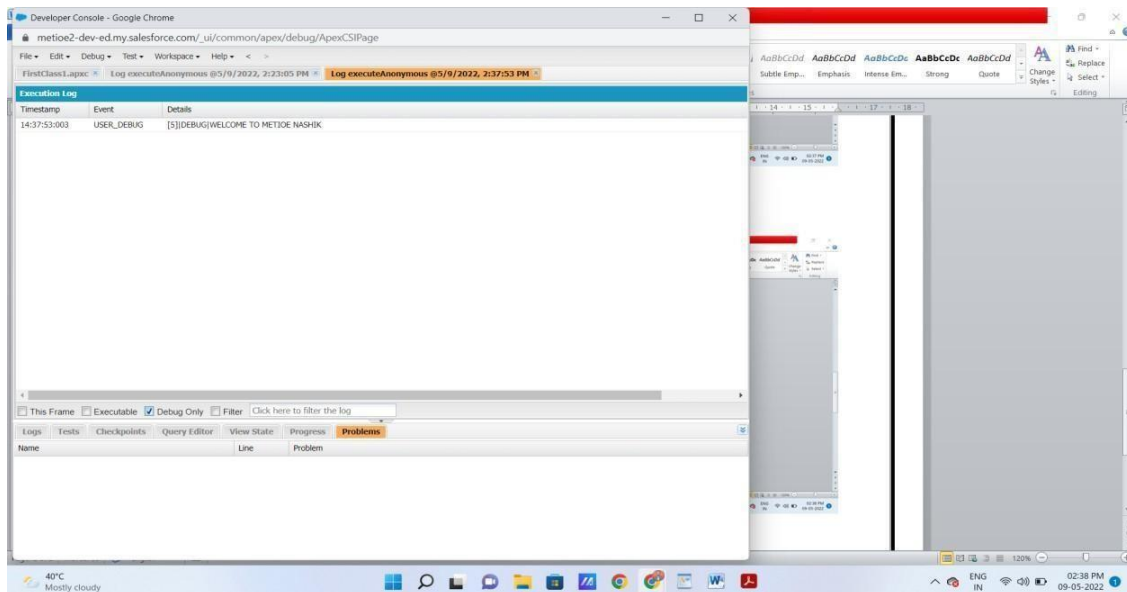


6. Click on Open log then Execute code

7. Click on Debug only (You will get output)



Output :



CC Practical - 4

Name : Ganesh Malaji Ghule

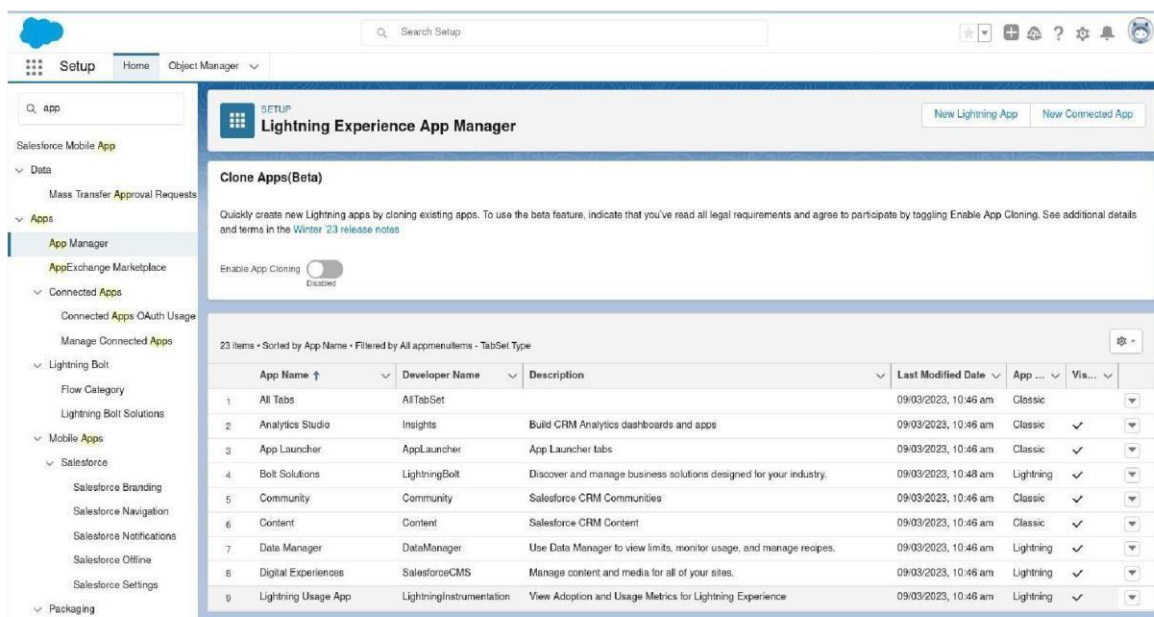
Roll No.:- 37

Div : TE-A

Title : Design and develop custom Application (Mini Project) using Salesforce Cloud.

STEPS TO CREATE AN APPLICATION (mini project) :

1. Open salesforce in Lightning Experience.
2. Open App Launcher ▪ View all ▪ Quick Search ▪ App Manager ▪ Click on New LightningApp.



3. fill the mention information.

The screenshot shows the 'New Lightning App' form, specifically the 'App Details & Branding' section. The form is divided into two main columns: 'App Details' and 'App Branding'. The 'App Details' column contains fields for 'App Name' (TECOA6), 'Developer Name' (TECOA6), and 'Description' (Enter a description...). The 'App Branding' column contains an 'Image' field with a placeholder image, a 'Primary Color Hex Value' field with a color picker and the value #0070D2, and a 'Org Theme Options' section with a checkbox 'Use the app's image and color instead of the org's custom theme' which is checked. At the bottom right, there is a 'Next' button.

4. App option (optional).

New Lightning App

App Options

Navigation and Form Factor

***Navigation Style**

☒ Standard navigation

☐ Console navigation

***Supported Form Factors**

☒ Desktop and phone

☐ Desktop

☐ Phone

Setup and Personalization

Setup Experience

☒ Setup (full set of Setup options)

☐ Service Setup

App Personalization Settings

☐ Disable end user personalization of nav items in this app

☐ Disable temporary tabs for items outside of this app

Next

5. Add Utility(optoinal).

New Lightning App

Utility Items (Desktop Only)

Give your users quick access to productivity tools and add background utility items to your app.

Add Utility Item

Utility Bar Alignment: **Default**

The utility bar is a fixed footer that opens components in docked panels. Available only when the app is viewed in Lightning Experience on a desktop.

Next

6.Next *Navigation Item (you can add item).

New Lightning App

Navigation Items

Choose the items to include in the app, and arrange the order in which they appear. Users can personalize the navigation to add or move items, but users can't remove or rename the items that you add. Some navigation items are available only for phone or only for desktop. These items are dropped from the navigation bar when the app is viewed in a format that the item doesn't support.

Available Items

CON

Communication Subscription Consents

Consumption Schedules

Contact Point Consent

Contact Point Type Consent

Contact Requests

Contract Line Items

Selected Items

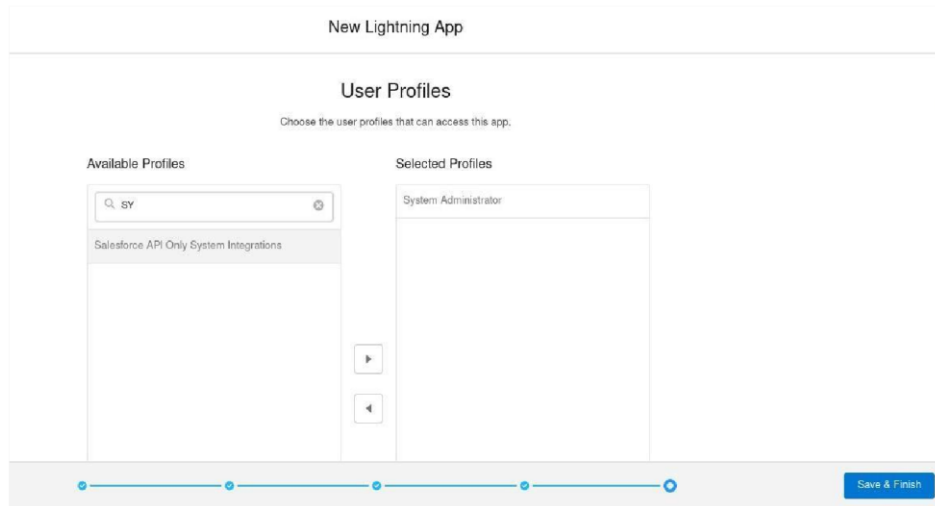
Calendar

Contacts

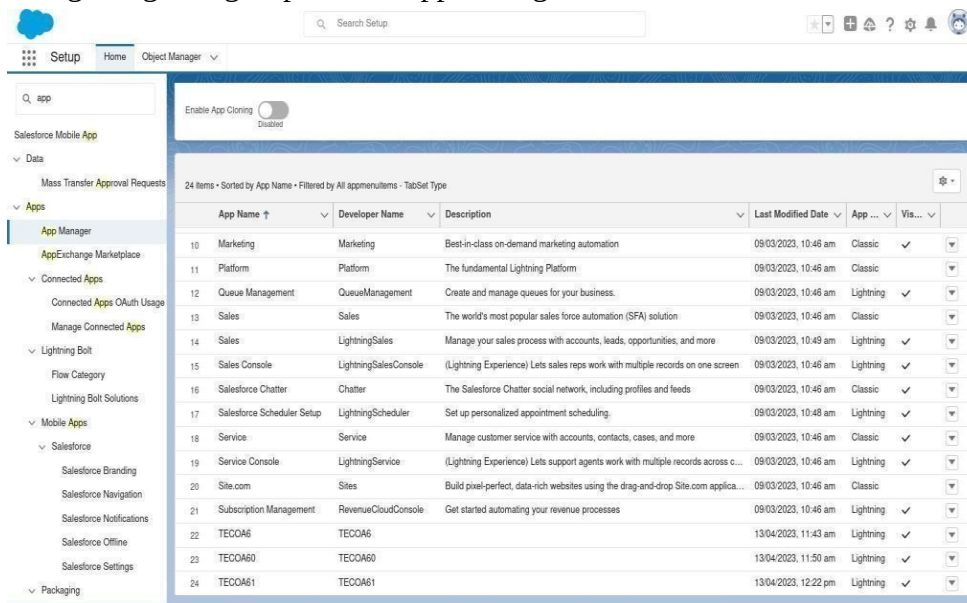
Back

Next

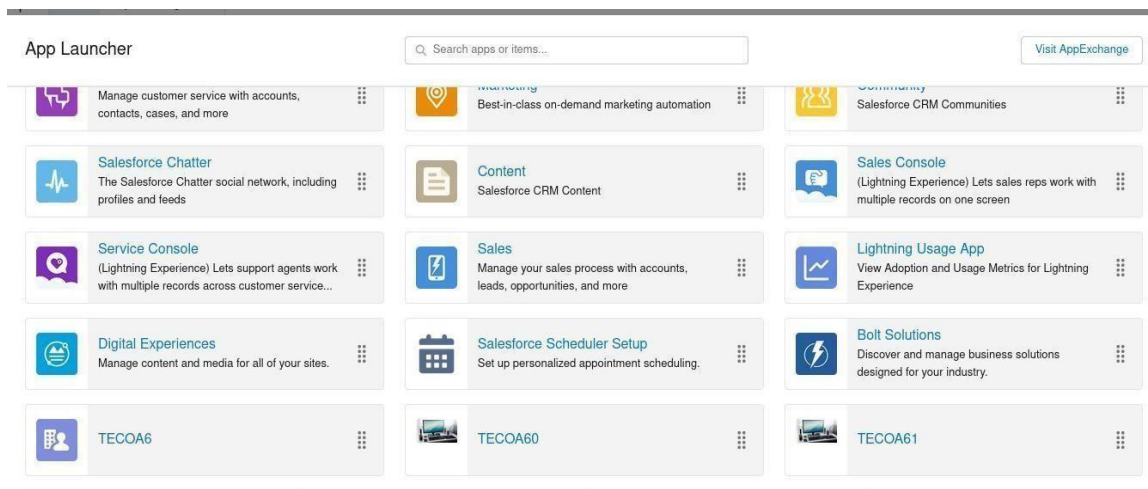
7. Next ▪ User Profile(use the user profilethat can access this app) ▪ Save and Finish.



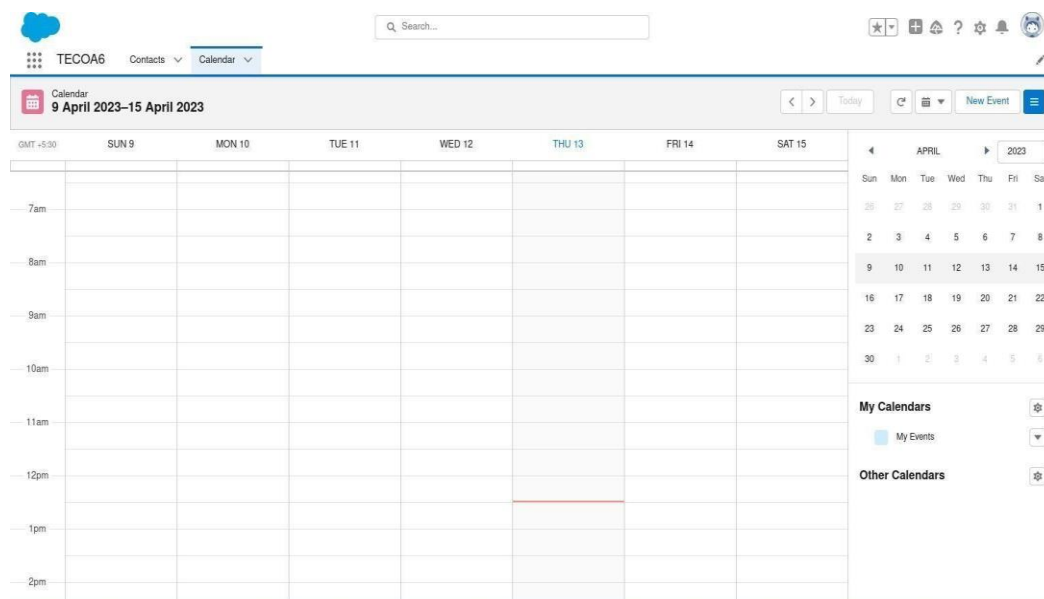
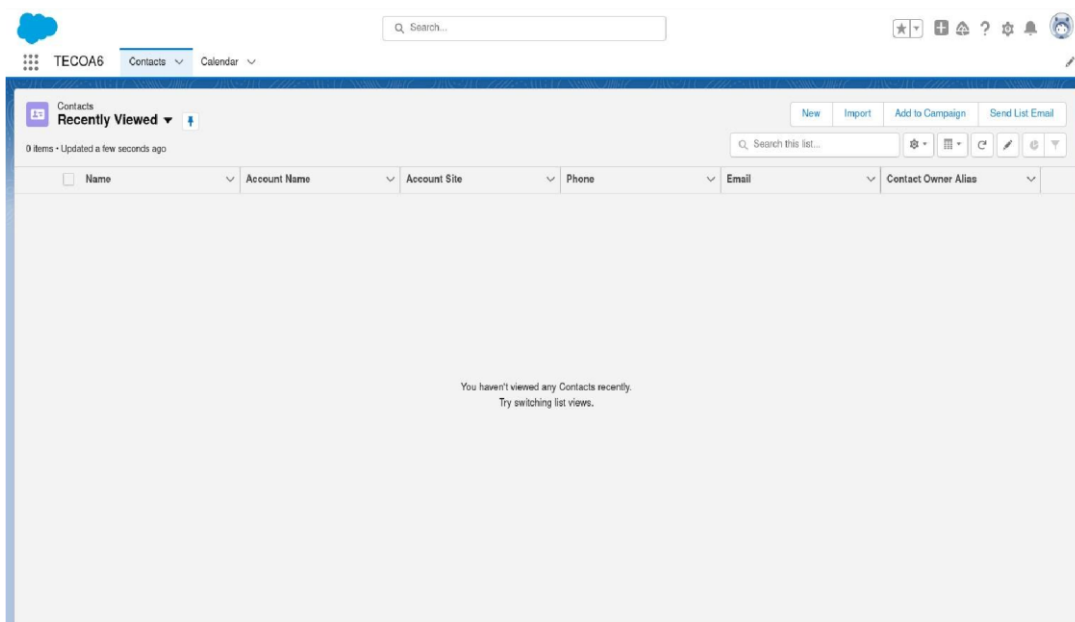
8. You will get Lightning Experience App Manager Window.



9. Search Your App by its name in App Launcher. just by entering some character, you can see app name with its logo.



10. Click on App ▪ you can see your application appearance in window ▪ Output.



AI Practical -1

Name:- Ganesh Malaji

Ghule Roll No:- 37

Div:- TE-A

Title:- Implement DFS and BFS Algorithm. Use an Undirected Graph and develop a Recursive Algorithm for searching all the vertices of the graph or tree data structure.

Program :-

Breadth First Search(BFS):-

Graph definition

```
graph = {  
    'A': ['B', 'C'],  
    'B': ['D', 'E'],  
    'C': ['F'],  
    'D': [],  
    'E': ['F'],  
    'F': []  
}
```

List to keep track of visited nodes

```
visited = []
```

Initialize a queue

```
queue = []
```

BFS function

```
def bfs(visited, graph, node):
```

```
    visited.append(node)
```

```
    queue.append(node)
```

```
    while queue:
```

```
        s = queue.pop(0)
```

```
        print(s, end=" ")
```

```
        for neighbour in graph[s]:
```

```
            if neighbour not in visited:
```

```
                visited.append(neighbour)
```

```
                queue.append(neighbour)
```

```
# Driver Code
bfs(visited, graph, 'A')
```

OUTPUT:

A B C D E F

2. Depth-first Search:

```
# Using a Python dictionary to act as an adjacency list
```

```
graph = {
    'A': ['B', 'C'],
    'B': ['D', 'E'],
    'C': ['F'],
    'D': [],
    'E': ['F'],
    'F': []
}
```

```
# Set to keep track of visited nodes
```

```
visited = set()
```

```
# Function for DFS
```

```
def dfs(visited, graph, node):
    if node not in visited:
        print(node, end=" ")
        visited.add(node)
        for neighbour in graph[node]:
            dfs(visited, graph, neighbour)
```

```
# Driver Code
```

```
print("Following is the Path using Depth-First Search:")
dfs(visited, graph, 'A')
```

OUTPUT:

Following is the Path using Depth-First Search:

A B D E F C

AI Practical-2

Name:- Ganesh Malaji Ghule

Roll No.:-37

Div:- TE-A

Title:- Implement A star Algorithm for any game search problem Program A* Algorithm.

Program:

```
from collections import deque
```

```
class Graph:
```

```
    def __init__(self, adjacency_list):
```

```
        self.adjacency_list = adjacency_list
```

```
    def get_neighbors(self, node):
```

```
        return self.adjacency_list.get(node, [])
```

```
# Heuristic function: here all values are equal (can be customized)
```

```
def h(self, node):
```

```
    H = {
```

```
        'A': 1,
```

```
        'B': 1,
```

```
        'C': 1,
```

```
        'D': 1
```

```
    }
```

```
    return H.get(node, float('inf'))
```

```
def a_star_algorithm(self, start_node, stop_node):
```

```
    open_list = set([start_node]) # Nodes to be evaluated
```

```
    closed_list = set()           # Nodes already evaluated
```

```
    g = {start_node: 0}           # Cost from start to current node
```

```
    parents = {start_node: start_node} # Parent tracking
```

```
    while open_list:
```

```
        # Find node in open_list with lowest  $f(n) = g(n) + h(n)$ 
```

```
        n = min(open_list, key=lambda x: g[x] + self.h(x))
```

```
        if n == stop_node:
```

```
            # Reconstruct path
```

```
            path = []
```



```
while parents[n] != n:
    path.append(n)
    n = parents[n]
path.append(start_node)
path.reverse()
print('Path found:', path)
return path
```

```
# For all neighbors of current node
for (m, weight) in self.get_neighbors(n):
    tentative_g = g[n] + weight
```

```
if m not in open_list and m not in closed_list:
    open_list.add(m)
    parents[m] = n
    g[m] = tentative_g
else:
    if tentative_g < g.get(m, float('inf')):
        g[m] = tentative_g
        parents[m] = n
        if m in closed_list:
            closed_list.remove(m)
        open_list.add(m)
```

```
open_list.remove(n)
closed_list.add(n)
```

```
print('Path does not exist!')
return None
```

```
# Example usage:
```

```
adjacency_list = {
    'A': [('B', 1), ('C', 3), ('D', 7)],
    'B': [('D', 5)],
    'C': [('D', 12)]
}
```

```
graph = Graph(adjacency_list)
graph.a_star_algorithm('A', 'D')
```

OUTPUT: Path found: ['A', 'B', 'D']

AI Practical-3

Name:- Ganesh Malaji Ghule

Roll No.:- 37

Div:- TE-A

Title:- Implement Greedy search algorithm for selection sort Program Greedy search Algorithm for following sort programs:

Program:

Selection Sort in Python

Time Complexity: $O(n^2)$

```
def selectionSort(array, size):
    for i in range(size):
        min_index = i

        # Find the index of the minimum element in the unsorted part
        for j in range(i + 1, size):
            if array[j] < array[min_index]:
                min_index = j

        # Swap the found minimum element with the first element
        array[i], array[min_index] = array[min_index], array[i]

# Driver Code
arr = [-2, 45, 0, 11, -9, 88, -97, -202, 747]
size = len(arr)

selectionSort(arr, size)

print("The array after sorting in Ascending Order by Selection Sort is:")
print(arr)
```

OUTPUT:

The array after sorting in Ascending Order by Selection Sort is:
[-202, -97, -9, -2, 0, 11, 45, 88, 747]

AI Practical -4

Name:- Ganesh Malaji Ghule

Roll No.:- 37

Div:- TE-A

Title:- Implement a solution for a constraint satisfaction problem using branch and bound and backtracking for n-queens problem or a graph coloring problem

Program :- n-queens problem

N-Queens problem using Backtracking

N = 4 # You can change this to solve for any N

```
def printSolution(board):
```

```
    for row in board:
```

```
        for val in row:
```

```
            print(val, end=" ")
```

```
        print()
```

Utility function to check if a queen can be safely placed at board[row][col]

```
def isSafe(board, row, col):
```

```
    # Check the current row on the left side
```

```
    for i in range(col):
```

```
        if board[row][i] == 1:
```

```
            return False
```

```
    # Check the upper diagonal on the left side
```

```
    i, j = row, col
```

```
    while i >= 0 and j >= 0:
```

```
        if board[i][j] == 1:
```

```
            return False
```

```
        i -= 1
```

```
        j -= 1
```

```
    # Check the lower diagonal on the left side
```

```
    i, j = row, col
```

```
    while i < N and j >= 0:
```

```
        if board[i][j] == 1:
```

```
            return False
```

```
        i += 1
```

```
        j -= 1
```

```
    return True
```

```

# Recursive utility to solve N-Queens problem
def solveNQUtil(board, col):
    # Base case: If all queens are placed
    if col >= N:
        return True

    # Try placing a queen in all rows in this column
    for i in range(N):
        if isSafe(board, i, col):
            board[i][col] = 1 # Place queen

            # Recur to place rest of the queens
            if solveNQUtil(board, col + 1):
                return True

            # If placing queen doesn't lead to a solution, backtrack
            board[i][col] = 0

    return False # If no place is possible in this column

# Main function to solve N-Queens problem
def solveNQ():
    board = [[0 for _ in range(N)] for _ in range(N)]

    if not solveNQUtil(board, 0):
        print("Solution does not exist")
        return False

    print("One of the solutions to the N-Queens problem is:")
    printSolution(board)
    return True

# Driver Code
solveNQ()

```

OUTPUT:

One of the solutions to the N-Queens problem is:

0 1 0 0

0 0 0 1

1 0 0 0

0 0 1 0

This means:

- Queen is at (0,1), (1,3), (2,0), (3,2)

AI practical -5

Name:- Ganesh Malaji Ghule

Roll No.:- 37

Div:- TE-A

Title:- Develop an elementary chatbot for any suitable customer interaction application
Program : Chatbot Program

```
def greet(bot_name, birth_year):
    print(f"Hello! My name is {bot_name}.")
    print(f"I was created in {birth_year}.")

def remind_name():
    print('Please, remind me your name.')
    name = input()
    print(f"What a great name you have, {name}!")

def guess_age():
    print('Let me guess your age.')
    print('Enter remainders of dividing your age by 3, 5, and 7.')

    rem3 = int(input())
    rem5 = int(input())
    rem7 = int(input())

    age = (rem3 * 37 + rem5 * 21 + rem7 * 15) % 105
    print(f"Your age is {age}; that's a good time to start programming!")

def count():
    print('Now I will prove to you that I can count to any number you want.')
    num = int(input())

    for i in range(num + 1):
        print(f"{i}!")

def test():
    print("Let's test your programming knowledge.")
    print("Why do we use methods?")
    print("1. To repeat a statement multiple times.")
    print("2. To decompose a program into several small subroutines.")
    print("3. To determine the execution time of a program.")
    print("4. To interrupt the execution of a program.")
```

```

answer = 2
while True:
    guess = int(input())
    if guess == answer:
        break
    print("Please, try again.")

print("Completed, have a nice day!")

def end():
    print("Congratulations, have a nice day!")
    print(". ")
    print(" .....")
    print(" .....")
    input()

# ----- Driver Code -----
greet('TE-Chatbot', '2022') # Change bot name and year as needed
remind_name()
guess_age()
count()
test()
end()

```

OUTPUT:

```

Hello! My name is TE-Chatbot.
I was created in 2022.
Please, remind me your name.
> Prathamesh
What a great name you have, Prathamesh!
Let me guess your age.
Enter remainders of dividing your age by 3, 5, and 7.
> 2
> 3
> 2
Your age is 23; that's a good time to start programming!
Now I will prove to you that I can count to any number you want.
> 5
0!
1!
2!
3!
4!

```

5!

Let's test your programming knowledge.

Why do we use methods?

1. To repeat a statement multiple times.
2. To decompose a program into several small subroutines.
3. To determine the execution time of a program.
4. To interrupt the execution of a program.

> 1

Please, try again.

> 3

Please, try again.

> 2

Completed, have a nice day!

Congratulations, have a nice day!

.

.

.

